HORIZONAJ · ENGENHARIA

Documentação Técnica

Sistema de Controladoria Municipal — arquitetura, ambiente, banco de dados, segurança, módulos, testes e operação

Guia para quem vai manter ou evoluir o sistema: como subir o ambiente, onde fica cada coisa, as regras de isolamento entre entidades, as integrações públicas e as rotinas de teste e documentação.

1. Visão geral e stack
2. Ambiente de desenvolvimento
3. Estrutura do projeto
4. Multi-entidade e segurança (RLS)
5. Autenticação e sessão
6. Módulos e rotas
7. Banco de dados e migrações
8. Integrações e alertas fiscais
9. Painel personalizável e preferências
10. Relatórios em PDF e Relatório Anual
11. IA, e-mails e rotinas agendadas
12. Testes
13. Manual, documentação e implantação
14. Convenções de código
15. Histórico das versões 1.1 e 1.2

HorizonAJ
Sistema de Controladoria Municipal

Versão 1.2
Outubro de 2026

1

Visão geral e stack

Aplicação web única (Next.js App Router) com PostgreSQL. Cada entidade (Prefeitura, Câmara, autarquia…) é um *cliente* com dados isolados no banco por Row Level Security.

Camada	Tecnologia	Observações
Framework	Next.js 16 (App Router, Turbopack, Server Actions), React 19, TypeScript estrito	Leia `node_modules/next/dist/docs/` antes de usar APIs novas (AGENTS.md).
Interface	Tailwind CSS 4, componentes shadcn/ui sobre Base UI, lucide-react, sonner	Seis temas em `src/lib/temas.ts` e `globals.css`.
Banco	PostgreSQL 17 + pgvector, Prisma 7 (adapter `@prisma/adapter-pg`)	Cliente gerado em `src/generated/prisma`.
Autenticação	Sessões próprias no banco, cookie assinado (jose), senhas Argon2 (`@node-rs/argon2`)	Ver capítulo 5.
Arquivos	S3 compatível (Silo/MinIO) ou disco local	`src/lib/armazenamento.ts`.
PDF	HTML → PDF com Playwright/Chromium	`src/lib/pdf/`, `src/lib/relatorios/`.
IA	OpenAI, Anthropic e Gemini (multiprovedor) com provedor "falso" para testes	`src/lib/ia/`.
E-mail	Nodemailer (Mailpit em desenvolvimento)	`src/lib/email/`.
Integrações	IBGE, SICONFI (Tesouro), Portal da Transparência (CGU); links do TCM-BA	`src/lib/integracoes/`.

```
Navegador ──► proxy.ts (cookie de sessão) ──► Server Components / Server Actions
                                                        │
                                  exigirContexto() ── sessão + vínculo + cliente ativo
                                                        │
                                comCliente(ctx, tx => …) ── SET app.cliente_id / app.usuario_id / app.perfil
                                                        │
                                  PostgreSQL (papel controladoria_app, sem BYPASSRLS) ── políticas RLS por cliente
```

2

Ambiente de desenvolvimento

Pré-requisitos

- Node.js 20+ (testado com 24) e npm.
- Docker Desktop (Postgres com pgvector, Redis, Silo/MinIO e Mailpit via `docker-compose.yml`). Sem Docker, `npm run db:local` sobe um Postgres embutido, **mas sem pgvector**: a migração `ia_documentos` falha nele, então use-o só para trabalho que não dependa da IA.
- Edge ou Chrome instalados (ou `npx playwright install chromium`) para PDFs e testes E2E.

Primeira execução

```
cp .env.example .env            # preencha senhas, SESSION_SECRET e CHAVE_CRIPTOGRAFIA
docker compose up -d            # Postgres, Redis, Silo e Mailpit
npm install                     # roda prisma generate no postinstall
npm run db:setup                # cria/atualiza o papel controladoria_app (antes das migrações)
npm run db:deploy               # aplica as migrações
npm run db:seed                 # normas, entidades e usuários de exemplo
npm run demo:popular            # (opcional) dados de demonstração de Catolândia/BA
npm run dev                     # http://localhost:3000
```

> **Porta 5432**
> Se o Postgres embutido (`db:local`) e o do Docker estiverem ativos ao mesmo tempo, o embutido escuta em `127.0.0.1:5432` e "rouba" as conexões de `localhost`. Pare um deles antes de rodar migrações.

Variáveis de ambiente

Variável	Uso
`DATABASE_URL`	Dono do banco: só Prisma CLI, setup, seed e scripts de teste.
`APP_DATABASE_URL`	Papel da aplicação, sujeito a RLS. Nunca use o dono no código da aplicação.
`SESSION_SECRET`	Assina o cookie de sessão.
`CHAVE_CRIPTOGRAFIA`	AES-256-GCM dos segredos gravados no banco (chaves de IA).
`STORAGE_*`, `S3_*`	Armazenamento de documentos.
`SMTP_*`, `EMAIL_REMETENTE`, `APP_URL`	E-mails de demandas e lembretes.
`PORTAL_TRANSPARENCIA_CHAVE`	Chave gratuita da API da CGU (vazia = transferências e CEIS/CNEP desabilitados).
`IA_PROVEDOR=falso`	IA determinística para testes, sem custo.
`PDF_CHROMIUM_PATH`	Chromium já instalado (opcional).

3

Estrutura do projeto

Pasta	Conteúdo
`src/app/(controle)/`	Telas da controladoria (perfis Controlador e Equipe), uma pasta por módulo: `painel`, `alertas`, `demandas`, `auditorias`, `autoavaliacao`, `planos`, `relatorios`, `dados-externos`... Cada módulo tem `page.tsx`, `actions.ts` (Server Actions) e componentes locais.
`src/app/satelite/`	Acesso restrito das unidades (demandas e painel da unidade).
`src/app/admin/`	Administração HorizonAJ: clientes, municípios, usuários, IA, trilha global.
`src/app/actions/`	Server Actions compartilhadas (sessão, evidências, **preferências**).
`src/components/`	`ui/` (shadcn), `shell/` (layout, menus por tema), componentes por domínio (`alertas/`, `demandas/`, `ia/`...) e `recolhivel.tsx`.
`src/lib/`	Regras e acesso a dados: `auth/`, `dados/` (consultas por módulo), `integracoes/`, `relatorios/`, `pdf/`, `ia/`, `email/`, `painel/`, `preferencias*.ts`, `risco.ts`, `rotulos.ts`, `temas.ts`.
`prisma/`	`schema.prisma`, migrações SQL (com RLS escritas à mão), seeds.
`catalogos/`	Normas em YAML (OT 05/2024 e Res. TCM-BA 1.120/2005), importadas por `npm run catalogos:importar`.
`scripts/`	Setup, seeds de demonstração, testes (`testar-*.ts`, `e2e-*.ts`), rotinas (lembretes, sincronização) e geração de documentos.
`docs/`	Manual do Controlador, esta documentação, implantação e notas de release.

Arquivos compartilhados entre servidor e cliente

Módulos usados por componentes `"use client"` não podem importar nada com `server-only`. Por isso alguns domínios têm dois arquivos: `preferencias-esquema.ts` (puro) e `preferencias.ts` (servidor); `anual-secoes.ts` / `anual-padrao.ts` (puros) e `anual.ts` (servidor).

4

Multi-entidade e segurança (RLS)

- **Tabelas globais** (sem RLS): `municipios`, `clientes`, `usuarios`, `sessoes`, `vinculos_cliente`, `normas`, `requisitos`. Acesse com `db` direto.
- **Tabelas por cliente**: têm `cliente_id`, `ENABLE` + `FORCE ROW LEVEL SECURITY` e a política `isolamento_cliente` (`cliente_id = app_cliente_id()`). Acesse sempre via `comCliente(ctx, tx => …)`, que abre transação e faz `set_config` de `app.cliente_id`, `app.usuario_id` e `app.perfil`.
- **Satélite**: políticas restritivas (`app_eh_satelite()`, `app_unidades_satelite()`) limitam o perfil às demandas das suas unidades; documentos internos (auditorias, alertas, relatório anual e seu texto padrão) recusam leitura e gravação.
- A aplicação conecta com `controladoria_app` (NOBYPASSRLS). `npm run test:rls` verifica o isolamento.

Checklist para uma tabela nova por cliente

1. Modelo no `schema.prisma` com `clienteId` e relação com `Cliente` (`onDelete: Cascade`).
2. Migração com `ENABLE` / `FORCE ROW LEVEL SECURITY`, política `isolamento_cliente` e, se for interna, a política restritiva do satélite; `GRANT` condicional ao papel (bloco `DO $$ … $$`), como em `20261008120000_painel_alertas_modelo_anual`.
3. CHECKs para JSON (`jsonb_typeof(...) = 'object'`) e faixas.
4. Casos em `testar-rls.ts` ou no teste do módulo (satélite e outro cliente não leem).

Perfis

Perfil	Escopo
`CONTROLADOR`	Tudo da entidade; decisões finais (concluir ciclo, validar ação, encerrar alerta, aparência, atualizar dados externos).
`EQUIPE`	Módulos da controladoria sem as decisões finais.
`SATELITE`	Somente demandas das unidades vinculadas (`/satelite`).
`adminHorizon`	Flag do usuário: administração global em `/admin`.

5

Autenticação e sessão

- `src/proxy.ts` (o antigo *middleware*) só verifica a presença/assinatura do cookie e redireciona para `/login?voltar=…`. A validação real da sessão é feita no servidor, no banco.
- `src/lib/auth/dal.ts`: `obterSessao` (cacheada por requisição), `exigirUsuario`, `obterContexto`, `exigirContexto(perfis)`. Toda página e Server Action começa por um deles.
- Link do município (`/m/<slug>`): a sessão guarda `municipioAcesso` e só alcança as entidades daquele município (admins HorizonAJ ficam livres).
- Tentativas de login são limitadas (`src/lib/limite.ts`) e registradas na trilha global.

6

Módulos e rotas

Os menus vêm de `src/components/shell/menus.ts`; o layout de navegação muda com o tema (`lateral`, `topo`, `trilho`, `faixas`) em `navegacao.tsx` / `app-shell.tsx`.

Rota	Módulo	Dados / regras
`/painel`	Painel personalizável (capítulo 9)	`lib/dados/painel.ts`, `lib/painel/blocos.ts`
`/dados-externos`	IBGE, SICONFI, Portal, TCM-BA	`lib/dados/integracoes.ts`, `lib/integracoes/`
`/normas`, `/autoavaliacao`	Normas e ciclos de autoavaliação	`lib/dados/autoavaliacao.ts`, `conformidade.ts`
`/planos`	Planos 5W2H (motor único)	`lib/dados/planos.ts`, `acoes.ts`
`/ia`	Sugestões da IA com revisão humana	`lib/ia/`
`/demandas`	Demandas às unidades	`lib/dados/demandas.ts`, `lib/demandas.ts`
`/auditorias`	PAAI, auditorias, checklists, achados	`lib/dados/auditorias.ts`
`/alertas`	Alertas (antigo “Medidas”): alertas fiscais + situações (modelo `Situacao`)	`lib/dados/alertas.ts`, `lib/integracoes/alertas-fiscais.ts`. `/medidas/*` redireciona (308) em `next.config.ts`.
`/documentos`	Arquivos e evidências	`lib/documentos.ts`, `lib/armazenamento.ts`
`/relatorios`	PDFs; `/relatorios/anual/padrao` e `/relatorios/anual/[ano]`	`lib/relatorios/`; rotas em `relatorios/pdf/*/route.ts`
`/unidades`, `/trilha`, `/aparencia`, `/configuracoes`	Gestão	`lib/auditoria.ts` (trilha), `lib/temas.ts`

> **Nomes internos × rótulos**
> O módulo foi renomeado para “Alertas” na interface, nas rotas e nos arquivos; o modelo do banco continua `Situacao` (tabela `situacoes`) e o enum `OrigemPlano.MEDIDA` foi mantido para não exigir migração de dados — o rótulo exibido é “Alerta” (`lib/rotulos.ts`).

7

Banco de dados e migrações

- Migrações em `prisma/migrations/AAAAMMDDHHMMSS_nome/migration.sql`; políticas RLS, CHECKs e GRANTs são escritos à mão no mesmo arquivo.
- Desenvolvimento: `npm run db:migrate` (cria) · Produção: `npm run db:deploy` (aplica). Depois de mudar o schema: `npm run db:generate`.
- UUID v7 (`@default(uuid(7))`), datas `timestamptz(3)`, nomes de colunas em snake_case via `@map`.

Migração da versão 1.1 — `20261008120000_painel_alertas_modelo_anual`

Objeto	Mudança
`usuarios.preferencias`	`JSONB NOT NULL DEFAULT '{}'` + CHECK de objeto. Preferências de exibição por usuário (tabela global, sem RLS).
`modelos_relatorio_anual`	Nova tabela por cliente (único por `cliente_id`), `secoes JSONB`, RLS de isolamento e bloqueio do satélite.

Migração aditiva, sem perda de dados e sem passo manual além do `db:deploy`.

8

Integrações e alertas fiscais

Fonte	O que coleta	Arquivo
IBGE	Município e população (censo/estimativa).	`integracoes/ibge.ts`
SICONFI	Extrato de entregas; RGF Anexo 1 (pessoal, RCL); RGF Anexo 2 (dívida); RREO Anexos 3 e 6 (RCL, resultado primário); RREO Anexo 14 (mínimos de educação, FUNDEB e saúde, orçamento e restos a pagar — os Anexos 8 e 12 não são publicados na API de dados).	`integracoes/siconfi.ts`
Portal da Transparência	Recursos federais por CNPJ, convênios, consulta CEIS/CNEP.	`integracoes/portal-transparencia.ts`
TCM-BA	Apenas links (sem API pública).	`integracoes/tcmba.ts`

A coleta roda por cliente (`sincronizar.ts`), grava em `coletas_integracao` e é disparada (1) pelo cron `npm run integracoes:sincronizar`, (2) pelo botão "Atualizar dados" e (3) automaticamente por `carregarDadosExternos()` quando o Painel, Alertas ou Dados externos abrem com dados de mais de 24 h (`VALIDADE_COLETA_MS`; após falha, nova tentativa só depois de 3 h). Os casos 2 e 3 usam `after()`; a trava `pg_advisory_xact_lock` + status `PROCESSANDO` evitam coletas simultâneas.

Dívida e resultado primário para a Câmara (v1.1)

Dívida consolidada e resultado primário são indicadores do ente, publicados pelo Executivo. Para clientes do Legislativo, `coletarSiconfi` agora lê o RGF Anexo 2 (com `co_poder=E`) e o RREO Anexo 6 a partir das entregas da Prefeitura, com o sufixo "— Executivo municipal" na referência. A despesa com pessoal continua sendo a do próprio Poder.

Alertas fiscais (`lib/integracoes/alertas-fiscais.ts`)

Funções puras que transformam a coleta do SICONFI em uma lista ordenada por gravidade (`EXCEDIDO` → `PRUDENCIAL` → `ALERTA` → `INFO`):

- **pessoal**: faixa da LRF ≠ regular (limites em `lrf.ts`: alerta 90%, prudencial 95%, máximo 54%/6%);
- **divida**: DCL/RCL ≥ 108% (alerta) ou > 120%;
- **entregas**: demonstrativos pendentes (prudencial se mais de dois);
- **coleta**: sem código IBGE, coleta falha ou dados com mais de 40 dias.

Também: **educacao**, **fundeb** e **saude** (aplicado < mínimo; `ALERTA` até o 5º bimestre, `EXCEDIDO` no 6º) e **resultado-primario** (abaixo da meta no último bimestre). Cada alerta traz `registro` (título, descrição com base legal, probabilidade e impacto), `referenciaAutomatica` (período do demonstrativo) e `doEnte` (indicador do município). A lista é exibida por `components/alertas/alertas-fiscais.tsx`, que mostra o atalho para o alerta automático aberto ou, na falta dele, o registro manual.

Alertas automáticos (v1.2 — `lib/integracoes/alertas-automaticos.ts`)

`executarSincronizacao` chama `aplicarAlertasAutomaticos(tx, …)` logo após gravar uma coleta do SICONFI com sucesso, numa transação própria (falha nos alertas não desfaz a coleta):

- **Criação**: para cada alerta sem situação aberta com a mesma `chave_automatica`, cria uma `Situacao` com origem `ALERTA`, `criado_por_id` nulo e número de `proximoNumeroSituacao()` (`lib/numeracao-situacao.ts`, compartilhado com a tela).

- **Atualização**: mudou a referência ou a gravidade → atualiza título, descrição, probabilidade e impacto.
- **Encerramento**: situação automática aberta cuja chave não está mais vigente → `RESOLVIDA`, `encerrado_por_id` nulo, justificativa automática.
- **Encerramento manual respeitado**: se existe situação encerrada pelo controlador com a mesma chave e a mesma `referencia_automatica`, não recria.
- **Câmara**: `alertasParaAutomatizar()` descarta alertas `doEnte` quando o Poder é o Legislativo.
- Trilha: `situacao.criada_automaticamente`, `…atualizada_automaticamente`, `…encerrada_automaticamente` (via `createMany`, como o log da coleta).
- E-mail: `lib/email/alertas.ts` monta a mensagem (modelo `modeloAlertasAutomaticos`) para controladores e equipe quando há alertas criados ou encerrados; `notificar: false` desliga (testes).
- Garantias no banco: índice único parcial `situacoes_automatica_aberta_key` (um aberto por chave) e CHECKs `situacoes_autor_ou_automatica` e `situacoes_chave_automatica_origem`. A tela impede trocar a origem de um alerta automático e reabrir um quando já há outro aberto para o mesmo indicador.

9

Painel personalizável e preferências

Catálogo de blocos

`lib/painel/blocos.ts` define os blocos (`id`, título, descrição e página de origem). A ordem do catálogo é a ordem de exibição. Para criar um bloco: adicione-o ao catálogo, ao mapa de ícones/largura em `painel/page.tsx`, ao `switch` de conteúdo e, se precisar de dados novos, a `resumoPainel`.

Preferências (`usuarios.preferencias`)

```
{
  "painel":         { "ocultos": ["unidades"], "recolhidos": ["indices"] },
  "dados-externos": { "recolhidos": ["fontes"] },
  "alertas":        { "recolhidos": ["painel"] }
}
```

- Esquema e leitura tolerante em `preferencias-esquema.ts` (zod; ids `[a-z0-9-]`, até 50). Guardamos **ocultos** (e não visíveis) para que blocos novos apareçam por padrão.
- `gravarPreferencia()` faz o merge numa única instrução SQL (`preferencias || jsonb_build_object(...)`), sem ler antes: cliques rápidos ou outra aba não sobrescrevem os demais campos.
- Server Actions em `app/actions/preferencias.ts`: `salvarRecolhidos(pagina, ids)` (sem revalidar) e `salvarBlocosPainel(visiveis)` (revalida `/painel`).

Componente `components/recolhivel.tsx`

- `ProvedorRecolhiveis` recebe a página e os recolhidos iniciais (lidos no servidor, sem “piscar”) e grava as mudanças com atraso de 400 ms.
- `BlocoRecolhivel`: cartão com cabeçalho-botão (`aria-expanded`/`aria-controls`), resumo quando recolhido e conteúdo renderizado no servidor (apenas `hidden`).
- `AlternarTodos`: “Recolher tudo / Expandir tudo”.
- Para usar em outra página, inclua o nome em `PAGINAS_PREFERENCIA`.

Menu sem barra de rolagem

O menu lateral foi compactado e as áreas que ainda podem rolar em telas baixas usam a utilidade `sem-barra-rolagem` (`globals.css`, `@utility`); os temas “topo” e “faixas” quebram linha em vez de rolar na horizontal. `npm run test:e2e:painel` verifica os seis temas em 1366×768 e 1366×650.

10

Relatórios em PDF e Relatório Anual

- Cada relatório tem um montador em `lib/relatorios/<tipo>.ts` que devolve HTML (template `html``` com escape automático), cabeçalho e rodapé; as rotas `relatorios/pdf/*/route.ts` chamam `responderPdf`, que gera o PDF e registra a emissão na trilha.
- O satélite é recusado em `exigirControle()`, além da RLS.

Texto padrão do Relatório Anual (v1.1)

Resolução de cada uma das seis seções (`anual-padrao.ts → secoesEfetivas`):

1. texto do exercício (`relatorios_anuais.secoes`);
2. texto padrão do cliente (`modelos_relatorio_anual.secoes`);
3. texto-base do sistema (`TEXTO_BASE_ANUAL`).

- Ao salvar um exercício, `somentePersonalizadas()` grava só as seções diferentes do padrão (comparação normalizando quebras de linha e espaços nas pontas). Assim, as demais seguem o padrão inclusive quando ele mudar.
- Ao salvar o padrão, `somenteDiferentesDoSistema()` não grava seções idênticas ao texto-base.
- Variáveis `{entidade}`, `{municipio}`, `{uf}`, `{ano}`, `{ano_seguinte}` são substituídas só na emissão (`aplicarVariaveis`); variáveis desconhecidas ficam como estão.
- Telas: `/relatorios/anual/padrao` (texto padrão + seletor de exercício) e `/relatorios/anual/[ano]`; o formulário `form-relatorio-anual.tsx` serve aos dois modos e mostra, por seção, se segue a referência.
- Trilha: `relatorio_anual.salvo` (com `secoesPersonalizadas`) e `relatorio_anual.padrao_salvo`.

11

IA, e-mails e rotinas agendadas

- **IA**: configuração por cliente/global em `/admin/ia` (chaves cifradas), mascaramento de dados pessoais antes do envio, citações obrigatórias e revisão humana das sugestões. `IA_PROVEDOR=falso` nos testes.
- **E-mails**: notificações de demandas (`lib/email/`); em desenvolvimento, caixa do Mailpit em `http://localhost:8025`.
- **Cron** (ver `docs/implantacao-vps.md`): `npm run lembretes:prazo` (lembretes de prazo) e `npm run integracoes:sincronizar` (dados externos, diário).

12

Testes

Os testes são scripts `tsx` com verificações explícitas (saída "OK/FALHA" e código de saída ≠ 0 em falha). Os que importam módulos `server-only` rodam com `--conditions=react-server` (já configurado no `package.json`). Requerem migrações + seed; os E2E requerem também `npm run dev`.

Comando	Cobre
`npm run typecheck` · `npm run lint`	TypeScript estrito e ESLint.
`test:rls`	Isolamento entre clientes e restrições do satélite.
`test:acesso-municipio`, `test:senha`	Link do município, sessão, política de senha.
`test:autoavaliacao`, `test:risco`, `test:auditorias`	Regras de conformidade, matriz de risco, ciclo da auditoria.
`test:relatorios`	Todos os PDFs, regras do texto padrão do relatório anual e RLS dos textos.
`test:preferencias`	Leitura tolerante, merge atômico e CHECK das preferências.
`test:alertas-automaticos`	Criação, idempotência, atualização, encerramento automático e manual, índice único, regra da Câmara, RLS, e-mail e coleta completa simulada com verificação no Mailpit.
`test:integracoes` (`-- --real`)	Parsing das APIs, prazos do SICONFI, limites da LRF, dívida para a Câmara, alertas fiscais; opcionalmente as APIs reais.
`test:ia`, `test:config-ia`, `test:emails`	IA com provedor falso, configuração e modelos de e-mail.
`test:e2e`, `test:e2e:demandas`	Fluxos principais no navegador (seed de exemplo).
`test:e2e:painel`	Painel personalizável, persistência dos blocos recolhidos, Alertas, redirecionamento de `/medidas`, texto padrão e menu sem barra de rolagem nos seis temas (dados de Catolândia).

Dados de teste

Os testes que dependem do seed escolhem os clientes mais antigos (`ORDER BY criado_em`), porque o banco pode conter também os dados de demonstração. Os testes que alteram registros (tema, preferências, texto padrão) restauram o estado original no `finally`.

13

Manual, documentação e implantação

Comando	Gera
`npm run manual:pdf -- --capturar`	Refaz as capturas (`scripts/manual/capturas.ts`, 1366×768, dados de Catolândia) e gera `docs/manual/Manual-do-Controlador.pdf` a partir de `manual-controlador.html`, com o sumário paginado automaticamente.
`npm run docs:tecnico`	Este documento: `docs/tecnico/Documentacao-Tecnica.pdf` a partir de `documentacao-tecnica.html`.
`docs/notas-de-release/`	Notas de release por versão (Markdown).
`docs/implantacao-vps.md`	Implantação com Docker Compose, proxy reverso (Caddy/OpenResty) e cron.

Publicar a versão 1.1

```
git pull
docker compose -f docker-compose.prod.yml build app
docker compose -f docker-compose.prod.yml run --rm app npm run db:deploy
docker compose -f docker-compose.prod.yml up -d app
```

14

Convenções de código

- Código, rotas, tabelas e mensagens em português; nomes descritivos (`exigirContexto`, `calcularAlertasFiscais`).
- Páginas são Server Components; interatividade em componentes `"use client"` pequenos. Mutação por Server Action que começa com `exigirContexto`/`exigirUsuario`, valida com zod, grava via `comCliente`, registra na trilha e chama `revalidatePath`.
- Retorno padrão das actions: `EstadoAcao` (`{ ok, erro, mensagem }`); mensagens de erro amigáveis via `mensagemDeErro`.
- Estados vazios, de carregamento e de erro explícitos nas telas; acessibilidade (rótulos, `aria-*`, foco visível).
- Rótulos exibidos ficam em `lib/rotulos.ts`; regras puras em módulos sem dependência de servidor, para serem testadas sem banco.
- Formatação: Prettier com largura de linha 150 (padrão dos arquivos existentes).

15

Histórico das versões

1.2 (16/10/2026) — alertas automáticos

Mudança	Arquivos principais
Criação, atualização e encerramento automáticos dos alertas	`lib/integracoes/alertas-automaticos.ts`, `lib/integracoes/sincronizar.ts`, `lib/numeracao-situacao.ts`, migração `20261016120000_alertas_automaticos`
Aviso por e-mail	`lib/email/alertas.ts`, `lib/email/modelos.ts`
Coleta automática ao abrir as telas	`lib/dados/integracoes.ts`
Saúde, educação, FUNDEB, orçamento e restos a pagar (RREO Anexo 14)	`lib/integracoes/siconfi.ts`, `tipos.ts`, `alertas-fiscais.ts`, `dados-externos/execucao.tsx`, `painel/blocos.tsx`
Telas ligadas ao alerta automático	`components/alertas/alertas-fiscais.tsx`, `lib/dados/alertas.ts`, `alertas/page.tsx`, `alertas/[id]/page.tsx`, `alertas/actions.ts`
Testes	`testar-alertas-automaticos.ts`, casos novos em `testar-integracoes.ts`

1.1 (08/10/2026)

Mudança	Arquivos principais
Medidas → Alertas (rótulos, rotas, PDF, trilha) com redirecionamento	`app/(controle)/alertas/`, `lib/dados/alertas.ts`, `lib/relatorios/alertas.ts`, `components/alertas/`, `next.config.ts`
Alertas fiscais automáticos (folha, dívida, SICONFI)	`lib/integracoes/alertas-fiscais.ts`, `components/alertas/alertas-fiscais.tsx`
Dívida/resultado primário também para a Câmara	`lib/integracoes/siconfi.ts`, `dados-externos/page.tsx`
Painel personalizável com índices em destaque	`app/(controle)/painel/*`, `lib/painel/blocos.ts`, `lib/dados/painel.ts`
Blocos recolhíveis lembrados por usuário	`components/recolhivel.tsx`, `lib/preferencias*.ts`, `app/actions/preferencias.ts`
Menu sem barra de rolagem em todos os temas	`shell/navegacao.tsx`, `shell/app-shell.tsx`, `globals.css`
Texto padrão do Relatório Anual	`lib/relatorios/anual-padrao.ts`, `relatorios/anual/*`, `lib/relatorios/anual.ts`
Testes	`testar-preferencias.ts`, `e2e-painel.ts`, casos novos em `testar-integracoes.ts` e `testar-relatorios.ts`; correções de espera em `e2e-smoke.ts`